Sprint 2 — Technical Design Document

Project 01B: Finding the Next Best Step

Team DSIders

Sprint 1 ended with a design: four services, a clickable Figma prototype and a worked example. Sprint 2 is where we built it. This document records what was built in the sprint, how it is put together (the technical design is the main part), and the places where the build departed from the plan.

1 Notes on this sprint

Issues were modified slightly. While building, a few of the issues we had planned turned out to be hard to do as written, so we adjusted them to what was feasible within the sprint. The scope and intent of the project are unchanged.

2 About the project and what this sprint delivered

Next Best Step is an Android app built for Barabari Collective. A student says what they want to become, answers a few questions written for them, and gets one concrete next step. When the app is not sure, it does not guess: it hands the student to a named Barabari caseworker. Caseworkers see who asked for help, can override any judgement the app made, and maintain the roles and job openings the app reasons over.

In Sprint 1 this was a design. In Sprint 2 it became a working application of about 7,000 lines of Kotlin, with these parts delivered:

Area	What was built
Project foundation	Android project (Kotlin, Jetpack Compose, Gradle), Room database schema with entities and DAOs, and a repository that is the single access point to data.
Requirement graph	Roles broken into ordered requirement steps with prerequisites, plus a repository loader. Seed data for three MVP roles and sample student profiles with some fields deliberately left blank.
The four engine services	S1 intake parser, S2 question picker, S3 gap reasoner, S4 confidence gate with mentor handover, and the orchestration that chains them into “next question / next step”.
Resume reading	Resume input as PDF, photo or pasted text, feeding the S1 intake parser.
Student app	Login and account creation, goal entry, personalised questions, “Where you stand”, “Your next step”, “Your path”, profile, help conversations and job openings (Figma frames 0–8).
Caseworker app	My students, student detail with an Override on every row, requests, jobs tab (roles and openings), caseworker profile and caseworker-led intake (Figma frames 9–11). The next step can be sent by WhatsApp click-to-chat or printed.
Quality	Test fixtures and goal sentences, and 17 engine unit tests including the Sprint 1 worked example. Demo personas, a government role, and the README.

Two additions go beyond the Sprint 1 design. A fourth caseworker tab, **Jobs**, was added because the Figma’s three tabs had no place to manage roles and openings. And for goals Barabari has not defined yet (for example “ML engineer”), the model drafts a path and saves it as an **unreviewed** role; it stays flagged *Needs review* until a caseworker checks it.

3 Technical design

3.1 Design principles

1. **The language model never decides what a student is missing.** It turns text into fields and phrases questions in the English Natural Language. Every gap comes from a deterministic check against the requirement graph that the NGO maintains. This is what makes the output explainable and reviewable.
2. **One question at a time, and stop early.** Questions are ranked by how much they would change the next step, and asking stops once nothing is worth asking (three or four taps in practice).
3. **Honest uncertainty.** A confidence score is tracked through the session. When the student needs help from a human, he/she can click on the message option and the app offers a named mentor.
4. **Works without a network key.** With no API key the app falls back to keyword matching and template questions.

3.2 System block diagram

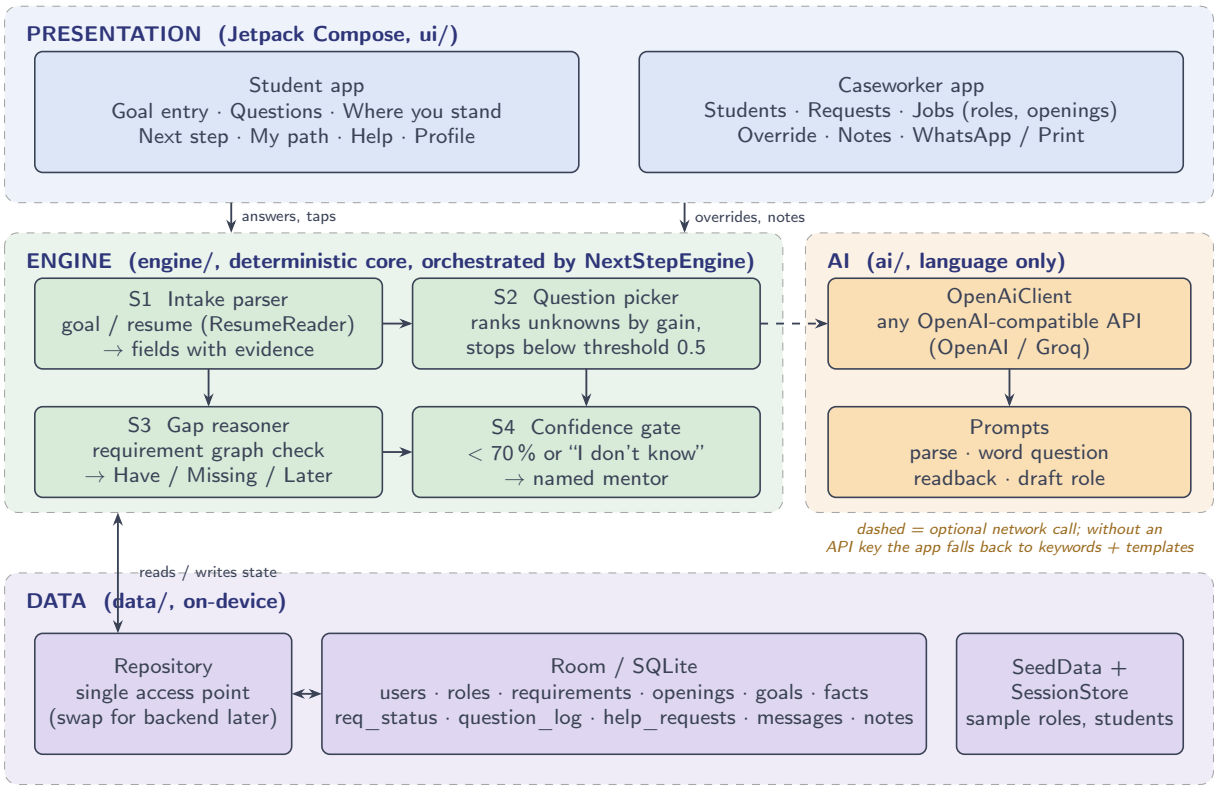


Figure 1: Block diagram. Solid arrows are in-app calls; the dashed arrow is the optional network call to a language model.

The app has four layers, mirrored in the package layout under `org.dsiders.nextstep`:

Layer	Package	Responsibility
Presentation	`ui/`	Compose screens for the student and caseworker paths, a Figma-derived theme and shared components. Screens only call the engine and repository; they hold no decision logic.
Engine	`engine/`	`NextStepEngine` orchestrates S1–S4. All decisions about gaps, next step and handover live here and are deterministic.
AI	`ai/`	`OpenAiClient` (any OpenAI-compatible endpoint, for example OpenAI or Groq) and `Prompts`. Used only for language tasks.
Data	`data/`	Room/SQLite entities and DAOs, `Repository` as the one access point, seed data and the session store.

3.3 The four engine services

Service	Input → output	How it works	Uses LLM?
S1 Intake parser	Goal sentence or resume → fields with evidence	Extracts only what is explicitly stated and attaches the evidence. Facts are stored with their source. Falls back to keyword matching with no key.	Yes
S2 Question picker	Student context → next question	Ranks unknown requirements by expected gain and returns the best one above the threshold (0.5). Only the wording of the question is personalised by the LLM.	Wording only
S3 Gap reasoner	Student context + role → Have / Missing / Later, path, next step	Orders the role's steps with prerequisites first, then finds the nearest reachable step whose own prerequisites are met.	No
S4 Confidence gate	Gap result → continue or hand over	Averages confidence over the steps that decide the next step. Below 0.70, or after "I don't know", a named mentor is offered with the reason.	No

3.4 Question picker (S2)

For each unknown core requirement the picker computes a gain, then asks the highest one. The terms are:

```
gain = uncertainty × position × (1 + 0.2 × dependents)

uncertainty  :  1.0 if never asked, else 1 − confidence
position     :  1.0 for the step that decides the next step (the ‘‘frontier’’)
                or any earlier step; 0.35 for steps after it
dependents   :  how many other core steps list this one as a prerequisite
```

Requirements that are already settled (confidence at least 0.6) or already asked are skipped, and the session ends when no candidate reaches the threshold of 0.5. A single answer can cover several requirements; those are written back as covered so the student is not asked twice. Each question carries a plain-language reason ("This decides your very next step"), shown through "Why are you asking this?".

3.5 Gap reasoner and confidence gate (S3, S4)

Requirements form a graph: each has an order, a reason it is needed, and prerequisites. The reasoner sorts them so prerequisites always come first (cycles are detected and do not hang), then splits the student's position into three groups, each with a reason: **Have**, **Missing** and **Not worth worrying about yet**. The next step is the first unmet core requirement whose prerequisites are met. Each step in the student's path is marked *Done*, *You are here* or *Upcoming*.

Rule	Value
Handover threshold	Confidence below 0.70, or the student said "I don't know"
A requirement counts as settled	Status known and confidence at least 0.60
Confidence when never asked	0.30
Unreviewed (AI-drafted) role	Confidence capped at 0.60, so a mentor is always offered
Caseworker override	Replaces the status on any row, for example pushing a step to "later"

The handover carries a named reason, such as "Not sure about: resume" or "New goal, path not reviewed", so the caseworker sees why the student was passed on.

3.6 Data model

Table	Purpose
`users`	Students and caseworkers, with role, language and availability
`roles`, `requirements`	The NGO-maintained requirement graph: a role's ordered steps, reasons and prerequisites; a reviewed flag on each role
`openings`	Real vacancies linked to a role
`goals`, `facts`	The student's stated goal and the facts extracted from what they said or uploaded
`req_status`	Per student and requirement: status, confidence, evidence and source (student, resume or caseworker)
`question_log`	What was asked and answered, so nothing is asked twice
`help_requests`, `messages`, `notes`, `activity`	Handover threads, caseworker notes and daily numbers

3.7 End-to-end runtime flow

1. The student enters a goal sentence, optionally with a resume. S1 parses it into fields with evidence and the app reads it back.
2. The engine matches the goal to a role. If none exists, the model drafts one and it is stored as unreviewed.
3. S2 picks the highest-gain unknown; the LLM words it using the student's course, college, earlier answers and language.
4. The answer updates `req_status`. S3 recomputes Have / Missing / Later and the next step.

5. If nothing is above the gain threshold, questioning stops. S4 checks confidence: at or above 0.70 the student sees one next step; otherwise a named mentor is offered.
6. A caseworker sees the student, may override any row, and can send the next step by WhatsApp or print it.

3.8 Technology choices

Choice	Reason
Kotlin, Jetpack Compose	Native Android, matches the shared-phone, low-RAM context; Compose lets the screens follow the Figma closely. Android 8.0 (API 26) and above.
Room / SQLite	Data stays on the device, which suits shared centre phones and intermittent connectivity.
OpenAI-compatible client	One client works with OpenAI or Groq by changing a base URL and model name; no vendor lock-in.
Deterministic engine	Keeps gap decisions testable and explainable. 17 JUnit tests cover the worked example, prerequisites, cycles, handover and overrides.

4 Decisions and limitations

- **Data is stored on the phone.** Students and caseworkers share one device, for example at a centre; a student on one phone cannot reach a caseworker on another. Moving to a shared backend later means replacing `Repository` only.
- **The API key is compiled into the APK.** Acceptable for a demo or pilot; before wider rollout, calls should go through a small server. The only file to change is `OpenAiClient`.
- **Login was added as the Figma frames describe.** Anyone can create a caseworker account on the device; an access code can be added if that is a concern.
- **Fixed UI text is English.** Questions, guidance and readbacks follow the student's language setting.
- **No colour means rejection.** Missing items are amber and nothing is red, as the design requires.
- **Seed roles need review.** The sample roles, requirements and openings mirror the Figma; Barabari staff should review them before use with real students.

5 Data setup and pipeline

5.1 What is in place

The app runs on a small, hand-built dataset that mirrors the Figma sample data. It is enough to exercise every path through the engine, but it is not yet large or varied enough to measure the engine at scale.

Data	What it contains and why
Requirement graph seed	Three MVP roles plus a government role, each broken into ordered steps with a reason and prerequisites. Loaded by `SeedData` into Room on first launch.
Sample student profiles	Students with some fields deliberately left blank, so the question picker has real unknowns to rank and ask about.
Demo personas and accounts	Debug-build students (`priya`, `sana`, `arjun`, `kiran`, `divya`, `mahesh`) and caseworkers (`ravi`, available; `lakshmi`, away), so both the normal path and the "mentor unavailable" path can be shown.
Openings	Sample vacancies linked to roles, shown to students and managed in the Jobs tab.
Test fixtures and goal sentences	Fixed inputs used by the 17 engine unit tests, including the Sprint 1 worked example.

5.2 How data moves through the app

1. `SeedData` writes roles, requirements, openings and sample users into Room on first launch.
2. Student input (goal sentence, resume, answers) passes through S1, which writes `facts` and `req_status` rows, each tagged with its evidence and source.
3. Every question asked and answered is written to `question_log`.
4. S3 and S4 read only from the `Repository`; caseworker overrides write back to `req_status` with the source set to caseworker.

6 Evaluation criteria

These are the criteria we will use to judge whether the app is doing its job. Targets are the team's proposed starting points and will be revised with Barabari after the first pilot.

Criterion	What it measures	How it is measured	Target
Next-step accuracy	Whether the app's next step matches what a caseworker would recommend	Caseworker labels the next step for each test persona without seeing the app's answer; compare	$\geq 80\,\%$ match
Evidence coverage	Every extracted fact has supporting evidence attached	Automatic check over `facts`	100 %
Questions per session	The app stops early and never repeats itself	Count from `question_log`	Median ≤ 4; 0 repeats
Handover quality	The app hands over when it should, and does not when it should not	Caseworker marks each persona "should hand over" or not; compute recall and precision	Recall $\geq$ 90 %, precision $\geq$ 70 %
Explainability	Every Have / Missing / Later row and every question has a reason	Automatic check over engine output	100 %
Offline fallback	The full flow works with no API key	Run every persona with the key removed	100 % complete, no crashes
Engine correctness	Ordering, prerequisites, cycles, gate and overrides behave as specified	`./gradlew :app:testDebugUnitTest`	All tests pass
Response time	Wait between answering and seeing the next question	Timed on a low-end phone, with and without the model	≤ 3 s

Handover recall is weighted above precision on purpose: sending a student to a caseworker who did not strictly need one costs a few minutes, while giving a confident wrong next step can cost the student an opportunity.

7 Baseline results

The baseline is version 0.1 of the app, as delivered at the end of this sprint, measured against the criteria above. It is the reference point that later sprints must improve on.

7.1 Baseline measurements

Criterion	Baseline result (v0.1)	Against target
Engine correctness	e.g. 17 / 17 unit tests pass	met
Next-step accuracy	3 demo personas match the caseworker's choice	met
Questions per session	median 3 questions	met
Handover quality	handovers that were correct / missed / unnecessary	met
Offline fallback	3 personas completed with no key	met
Response time	seconds with model	met

7.2 What worked

- **The deterministic engine.** S2–S4 reproduce the Sprint 1 worked example, and the unit tests cover prerequisites, cycles, handover and overrides. Keeping the model out of gap decisions made these behaviours testable.

- **End-to-end flow.** A student can go from goal sentence to one next step, and a caseworker can see, override and send that step, on a single device.
- **Graceful fallback.** The app still completes the flow without an API key, using keyword matching and template questions.
- **Provider-neutral AI client.** Switching between OpenAI and Groq needs only a base URL and model name.
- **Unknown goals are handled safely.** AI-drafted roles are flagged *Needs review* and always offer a mentor, so an unreviewed path is never presented as certain.

7.3 What did not work, or not yet

- **Single-device data.** A student and a caseworker on different phones cannot reach each other.
- **Scope adjustments.** Some planned issues were reduced to what was feasible within the sprint (Section 1).

8 Implementing the hardest component first

We treated the **deterministic reasoning core** (S2 question picker and S3 gap reasoner, with the S4 gate on top) as the hardest component. Everything else in the app depends on it, it must be explainable to a caseworker, and by design it cannot lean on the language model to cover mistakes. The difficult parts were ordering requirements so prerequisites always come first, detecting cycles without hanging, defining a gain that picks the question that actually changes the next step, and a stopping rule that keeps sessions to three or four taps.

We de-risked it in this order:

1. The engine was written as plain Kotlin in `engine/`, with no dependency on Android or the UI, so it could be tested on its own.
2. The Sprint 1 worked example was encoded as a unit test, giving a fixed expected answer to build towards.
3. Tests for prerequisites, cycles, handover and overrides were added, bringing the total to 17.
4. The screens were then built on top. Because screens hold no decision logic, UI changes could not break the engine's behaviour.

The second-riskiest component, reading messy resumes (PDF, photo or pasted text), was isolated behind `ResumeReader` and S1, with keyword matching as a fallback, so a weak parse lowers confidence and triggers a handover instead of producing a wrong next step.

9 Preparing for Hack 1

Hack 1 should be a focused build that moves the baseline numbers, not a broad feature push. We propose the following plan, to be confirmed with our mentor.

Goal. Make the app measurable, then improve the weakest measured criterion.

Hack 1 focus	Outcome
Synthetic data pipeline	Generator from Section 5.3 producing a labelled set of profiles, goal sentences and resumes.
Evaluation harness	One script that runs every profile through the engine and prints the criteria table from Section 6, so each change can be compared with the baseline.
Fix the weakest criterion	Target whichever baseline number is furthest from its target (likely parser accuracy or the keyword fallback).
Seed data review	Barabari staff review the sample roles and openings so accuracy is judged against real requirements.

Ready going in: the dummy v0.1 app, the 17 engine tests, demo accounts, the evaluation criteria and the baseline from Section 7.

Success measure: the evaluation harness runs on the full synthetic set, and at least one criterion improves over the v0.1 baseline.

10 Running and testing

Open `NextBestStep/` in Android Studio, add an API key to `local.properties` (optional) and run on a phone or emulator. Debug builds include sample student and caseworker accounts. Engine tests run with `./gradlew :app:testDebugUnitTest`. Setup details are in the README.